Análise de Desempenho de Invocação Remota de Objetos (Web Services e Java RMI)

Francisco Duro (66640), Renato Mendes (65384)
Escola de Ciência e Tecnologia – UTAD
Mestrado em Engenharia Eletrotécnica e de Computadores

Resumo - Estudo da influência que a tecnologia *middleware* (*Web Services e Java RMI*), a quantidade de dados a transmitir e a largura de banda disponível têm no desempenho da execução de métodos remotos. Na implementação dos *Web Services* foram utilizados os protocolos de comunicação SOAP (com *Web Server* e sem *Web Server*) e REST (JSON).

Keywords – Redes, Sistemas Distribuídos, Web Services, Java RMI, SOAP, XML, REST, JSON, Web Server, Middleware, Invocação Remota

I. Introdução

Com a evolução das tecnologias de redes de dados surgiram os Sistemas Distribuídos. Um Sistema Distribuído, descreve um sistema que, ao contrário de um Sistema Centralizado, permite que vários nós de processamento, estejam ligados entre si e que haja uma distribuição de tarefas com recurso a diversas máquinas espalhadas pela rede.
A descentralização das tarefas de computação trouxe então inúmeras vantagens:

- Partilha física de recursos distribuídos;
- Soluções mais rápidas com a combinação de vários computadores;
- Aumento do grau de segurança e fiabilidade;
- Aumento de desempenho;
- Aumento da disponibilidade;
- Decréscimo do tempo de resposta.

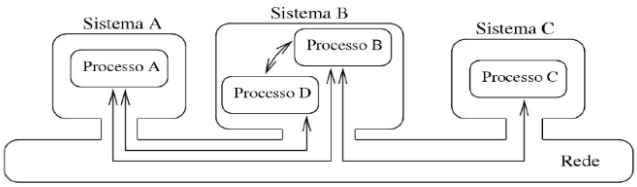


Figura 1 – Sistemas Distribuídos.

Assim, e contrariamente à computação Local, na computação Distribuída, podemos encontrar várias plataformas de hardware, vários Sistemas Operativos, vários ambientes de Execução e ainda inúmeras tecnologias e linguagens de programação.
Outra das grandes diferenças entre Sistemas Centralizados e Sistemas Distribuídos é a invocação local e a invocação remota de objetos, onde existe uma diferença na latência entre os dois métodos de invocação, sendo a invocação remota mais lenta.
Na invocação remota, os dados e os pedidos têm de viajar pela rede e estão sujeitos a diversos fatores (largura de banda, técnica de acesso ao meio, % de ocupação do canal, % de erros, etc.), dependendo da tecnologia utilizada para o envio dos mesmos.
Como existe um grande diversidade de plataformas, é então necessário o uso de tecnologias de *middleware*. O *middleware*, funciona como uma camada de abstração para o programador e tem como finalidade esconder detalhes e especificações dos sistemas de forma a oferecer uma visão uniforme tanto da rede como dos Sistemas Operativos.
No presente artigo o caso de estudo foi baseado em dois tipos de tecnologias de *middleware*, nomeadamente, *WebServices e Java RMI*, tendo como objetivo analisar o seu desempenho.

II. Invocação Local e Remota

A invocação de objetos refere-se a programas que chamam objetos que podem estar ou não na mesma máquina (local ou remota).
Na invocação local são conhecidas todas as características, tanto a nível de *hardware* como de *software*.
Na invocação remota é necessário um intermediário que permita fazer a compreensão do método por qualquer hardware ou software que seja diferente do método criado. Neste trabalho, *WebServices e Java RMI* foram as tecnologias de *middleware* utilizadas que permitem que o código funcione corretamente em máquinas diferentes.

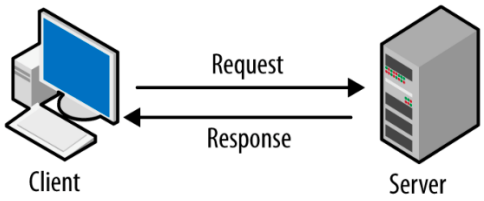


Figura 2 – Invocação Remota.

III. Web Services

Tecnologia *middleware* utilizada por plataformas distribuídas com o objetivo de transmitir dados através de protocolos de comunicação (SOAP e REST), independentemente de linguagens de programação, permitindo assim aceder e invocar métodos em máquinas distintas.

A. SOAP (Simple Objetct Access Protocol)

Protocolo baseado em XML como formato de mensagens, que permite transmissão de informação e dados entre Sistemas Distribuídos. Para a invocação remota de um método, é necessário especificar o endereço do componente, o nome do método e os argumentos para esse método. Os dados são enviados normalmente através do protocolo HTTP. Possibilita a comunicação entre aplicações que correm em diferentes sistemas operativos com diferentes linguagens de programação. Por esta razão, o SOAP tornou-se uma norma para se utilizar com *Web Services*, tecnologia baseada em XML e HTTP.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

```xml
<xml version="1.0" encoding="UTF-8">
<alunos>
   <aluno>
      <nome>Francisco</nome>
      <numero>66640</numero>
   </aluno>
   <aluno>
      <nome>Renato</nome>
      <numero>65384</numero>
   </aluno>
</alunos>
</xml>
```

Figura 3 – Exemplo do formato de representação de dados XML.

B. *REST (Representational State Transfer)*

Protocolo com estilo de arquitetura baseado no protocolo HTTP, com o objetivo de simplificar o acesso aos *Web Services*, definindo um conjunto de restrições a serem usadas para a criação dos mesmos. As restrições partem do cliente e são respondidas pelo servidor.

Este tipo de arquitetura utiliza como padrão para a representão dos dados, o formato de texto JSON (JavaScript *Object Notation*), mas também suporta XML, RSS, entre outros, fornecendo flexibilidade. O formato JSON foi criado para ser uma linguagem de transmissão de dados e, sendo muito mais rápida do que o XML acabou por se tornar uma das escolhas mais óbvias no que toca a formatos de representação de dados para transmissão.

```json
{
 "alunos":[
  {
   "nome": "Francisco",
   "numero": "66640"
  },
  {
   "nome": "Renato",
   "numero": "65384"
  }
 ]
}
```

Figura 4 - Exemplo do formato de representação de dados JSON.

IV. JAVA RMI

RMI (Remote Method Invocation), tecnologia *middleware* suportada pela plataforma Java. Tal como os Web Services, o Java RMI permite ao programador invocar métodos de objectos remotos ou locais, contudo estes apenas podem estar alojados em máquinas virtuais Java (JVMs) distintas, não suportando outras linguagens de programação.

Esta tecnologia é composta por Stubs do lado do cliente e Skeletons do lado servidor e, para que o cliente possa invocar um método de um objeto no servidor, o cliente tem de conhecer a interface que o objeto implementa e os métodos que o servidor disponibliza.

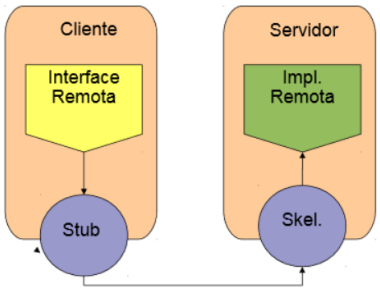


Figura 5 - Funcionamento do Java RMI.

O Java RMI utiliza mecanismos Java de Serialização para transmissão de dados, transformando o objeto numa sequência de bytes, que pode ser enviado e posteriormente des-serializado e reconstituido, numa JVM remota distinta.

V. CASOS DE ESTUDO

Análise de desempenho de invocação remota de objetos com diferentes tecnologias *middleware (Web Services e Java RMI).*

O presente artigo passa pela comparação da influência da tecnologia *Web Services* (REST e SOAP) e *Java RMI*, quantidade de dados a transmitir e largura de banda disponível, de forma a estudar em qual tecnologia, a transmissão de dados é mais eficaz.

Foram então realizados diversos ensaios, com diferentes larguras de banda e diferentes tamanhos de pacotes.

A. *Cenário da rede*

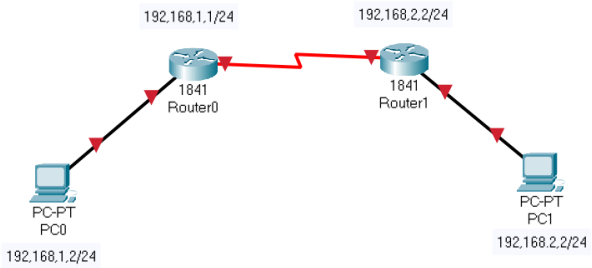


Figura 6 – Cenário de rede dos casos de estudo.

De modo a testar o desempenho das duas tecnologias simulou-se um Sistema Distribuído com recurso ao programa VirtualBox. Foram então criadas duas máquinas virtuais (PC0-Servidor e PC1-Cliente) com Sistema Operativo Linux (Lubuntu) e para que ambas conseguissem comunicar, uma terceira máquina com o WANEM (*Wide Area Network Emulator*) para simular os dois routers.

Como podemos ver na figura acima, estabeleceu-se o servidor (PC0) com o IP 192.168.1.2 e o cliente (PC1) com o IP 192.168.2.2.

B. *SOAP (XML)*

A implementação do *Web Service* SOAP, tem a particularidade de poder ser realizada de duas maneiras, com *Web Server* e sem *Web Server*.

1. *Servidor com Web Server*

Foi criada uma *Web Application* com um *Web Service Client* (SOAP), com um simples método que apenas recebe um array de bytes e imprime uma mensagem.

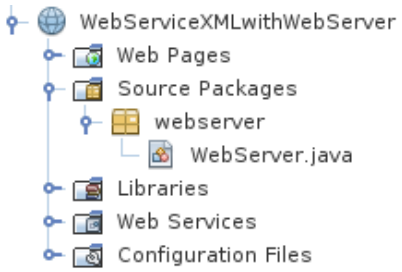


Figura 7 - NetBeans Package do Servidor com Web Server.

```
package webserver;

import javax.jws.WebMethod;
import javax.jws.WebService;

@WebService
public class WebServer {

    @WebMethod
    Public void process1(byte[] b){
           System.out.println("In Server 1");
    }
}
```

Código 1 – Processo do Servidor com Web Server.

2. *Servidor sem Web Server*

Para o Servidor sem *Web Server*, em vez da utilização de uma *Web Application* foi usado um *endpoint* para publicar o serviço no endereço escolhido.



Figura 8 - NetBeans Package do Servidor sem Web Server.

```
package webservicexmlwithoutwebserver;

import javax.xml.ws.Endpoint;

public class WebServiceXMLwithoutWebServer {

    public static void main(String[] args) {
           NoWebServer nws = new NoWebServer();
           Endpoint.publish("http://192.168.1.2:1234/
        NoWebServer", nws);
    }
}
```

Código 2 – Endpoint do Servidor sem Web Server.

```
package webservicexmlwithoutwebserver;

import javax.jws.WebService;

@WebService
public class NoWebServer {

    public void process2(byte[] b){
           System.out.println("In Server 2");
    }
}
```

Código 3 - Processo do Servidor sem Web Server.

3. *Cliente*

É o cliente quem faz a invocação dos métodos remotos localizados nos servidores (com e sem Web Server) e mede o tempo antes e depois do mesmo ser invocando, registando os seus valores.

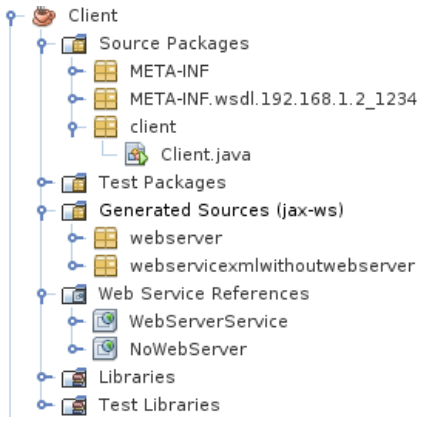


Figura 9 - NetBeans Package do Cliente SOAP.

```
package client;

import java.io.FileWriter;
import java.io.IOException;
import java.util.Random;
import java.util.logging.Level;
import java.util.logging.Logger;

public class Client {

    public static final int MAX_ITERATIONS = 7;
    public static final int MAX_OBJECT_INVOCATIONS =
   20;
    public static FileWriter
   objectInvocationPerformance = null;

    public static void main(String[] args) throws
   IOException{
        getObjectInvocationPerformanceAnalysis();
    }

    private static void
   getObjectInvocationPerformanceAnalysis() {
        try {
           objectInvocationPerformance = new
         FileWriter("WebServerInvocation.txt");
           objectInvocationPerformance.write("Time,
         Packet size\n");
           checkObjectInvocationPerformance();
```

```
        } catch (IOException ex) {
            Logger.getLogger(Client.class.getName())
          .log(Level.SEVERE, null, ex);
        } finally {
            try {
            objectInvocationPerformance.close();
            System.out.println("TXT file created
         succesfully!");
         } catch (IOException ex) {
            Logger.getLogger(Client.class.getName()).lo
         g(Level.SEVERE, null, ex);
            }
        }
    }

    private static void
checkObjectInvocationPerformance() throws IOException{
        int size = 1;
        for (int i=0; i < MAX_ITERATIONS; i++){
            byte[] dataPackets = new byte[size];
            setDataPackets(dataPackets);
            System.out.println(size + "\n");
            long time =
        getElapsedTimeOfObjectInvocation(dataPackets);
            objectInvocationPerformance.write(time +
        "," + size + "\n");
            size = size*10;
        }
    }

    private static void setDataPackets(byte[]
dataPackets) throws IOException{
        Random random = new Random();
        random.nextBytes(dataPackets);
    }

    private static long
getElapsedTimeOfObjectInvocation (byte[] dataPackets)
throws IOException{
        long elapsedTimeCounter = 0;
            for (int j=0; j < MAX_OBJECT_INVOCATIONS;
        j++){
                long startTime =
            System.currentTimeMillis();
                process1(dataPackets);
                long stopTime =
            System.currentTimeMillis();
                long elapsedTime = stopTime-startTime;
                System.out.println(elapsedTime);
                elapsedTimeCounter =
            elapsedTimeCounter + elapsedTime;
            }
        System.out.println("");
        long averageElapsedTime =
    elapsedTimeCounter/MAX_OBJECT_INVOCATIONS;
        return averageElapsedTime;
    }

    private static void process1(byte[] arg0) {
        webserver.WebServerService service = new
    webserver.WebServerService();
        webserver.WebServer port =
    service.getWebServerPort();
        port.process1(arg0);
    }
```

```
    private static void process2(byte[] arg0) {
        webservicexmlwithoutwebserver
    .NoWebServerService service = new
        Webservicexmlwithoutwebserver
    .NoWebServerService();
    webservicexmlwithoutwebserver.NoWebServer port
    = service.getNoWebServerPort();
        port.process2(arg0);
    }
}
```

Código 4 – Cliente SOAP.

C. *REST (JSON)*

1. *Servidor*

Tal como no SOAP, para o servidor REST também foi criada uma *Web Application*, mas desta vez com um *RESTful Web Service*. O processo disponibilizado é também parecido ao do SOAP, mas recebe uma string em vez de um array de bytes.

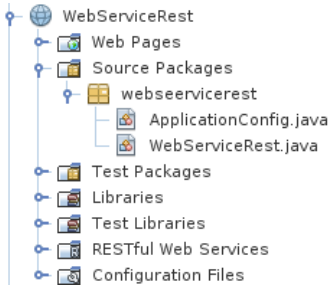


Figura 10 - NetBeans Package do Servidor REST.

```
package webseeservicerest;

import javax.ws.rs.core.Context;
import javax.ws.rs.core.UriInfo;
import javax.ws.rs.Consumes;
import javax.ws.rs.Produces;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.PUT;

@Path("WebServiceRest")
public class WebServiceRest {

    @Context
    private UriInfo context;

    public WebServiceRest() {
    }

    @GET
    @Produces(javax.ws.rs.core.MediaType.
  APPLICATION_JSON)
    public String getJson() {
          throw new UnsupportedOperationException();
    }

    @PUT
    @Consumes(javax.ws.rs.core.MediaType.
  APPLICATION_JSON)
    public void putJson(String content) {
        System.out.println("In Server 4");
    }
}
```

Código 5 - Servidor REST.

2. *Cliente*

Tal como no cliente SOAP, é o cliente quem faz a invocação e o registo dos resultados dos método remotos localizados no servidor.
A única alteração no código do cliente SOAP (Código 4), ocorre na função getElapsedTimeOfObjectInvocation que é adaptada ao REST.

```
private static long getElapsedTimeOfObjectInvocation
(byte[] dataPackets) throws IOException{
     NewJerseyClient service = new NewJerseyClient();
     Gson gson = new Gson();
     long elapsedTimeCounter = 0;
     for (int j=0; j < MAX_OBJECT_INVOCATIONS;
   j++){
           String data = gson.toJson(dataPackets);
           long startTime = System.currentTimeMillis();
           service.putJson(data);
           long stopTime = System.currentTimeMillis();
           long elapsedTime = stopTime - startTime;
           System.out.println(elapsedTime);
           elapsedTimeCounter = elapsedTimeCounter
        + elapsedTime;
     }
     System.out.println("");
     long averageElapsedTime =
   elapsedTimeCounter/MAX_OBJECT_INVOCATIONS;
     return averageElapsedTime;
}
```

Código 6 - Excerto do código do Cliente REST.

D. *Java RMI*

1. *Servidor*

Para o servidor RMI, respeitou-se o modelo da Figura 5 e criou-se a interface que o cliente RMI precisa de usar, o serviço que este servidor implementa e a classe do Servidor onde se cria o Skeleton. Tal como no SOAP o processo disponibilizado pelo servidor, apenas recebe um array de bytes e imprime uma mensagem.

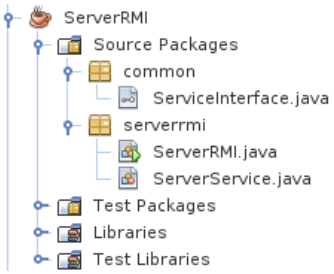


Figura 11 – NetBeans Package do Servidor RMI.

```
package common;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ServiceInterface extends Remote{
    public void process3(byte[] b) throws
RemoteException;
}
```

Código 7 – Iterface Remota RMI.

```
package serverrmi;

import common.ServiceInterface;

public class ServerService implements ServiceInterface
{
    @Override
    public void process3(byte[] b){
         System.out.println("In Server 3");
    }
}
```

Código 8 – Implementação Remota RMI.

```
package serverrmi;

import java.rmi.AlreadyBoundException;
import java.rmi.Remote;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;
import java.util.logging.Level;
import java.util.logging.Logger;

public class ServerRMI extends ServerService {

    public static void main(String[] args) {
        try {
            ServerRMI serverRMI = new ServerRMI();
            System.setProperty("java.rmi.server.
        hostname", "192.168.1.2");
            Remote proxy =
         UnicastRemoteObject.exportObject(serverRMI,0)
         ;
            Registry registry =
        LocateRegistry.createRegistry(1234);
            registry.bind("Server Service", proxy);
            System.out.println("Server is Online!");
        } catch (RemoteException |
    AlreadyBoundException ex) {
           Logger.getLogger(ServerRMI.class.getName())
         .log(Level.SEVERE, null, ex);
        }
    }
}
```

Código 9 – Servidor RMI e criação do Skeleton.

2. *Cliente*

Tal como no cliente SOAP e REST, é o cliente quem faz a invocação e o registo dos resultados dos método remotos localizados no servidor. Contudo, no Cliente RMI, é preciso a Interface que especifica os métodos que o Servidor implementa (Código 7) e ainda uma classe chamada ClientRMI, onde criamos o Stub para comunicar com o Skeleton do Servidor. Depois temos novamente uma classe chamada Client que é parecida à classe utilizada no Cliente SOAP e Cliente REST, onde a única alteração no código, relativamente ao código do cliente SOAP (Código 4), ocorre na função *getElapsedTimeOfObjectInvocation* que é adaptada ao RMI e no acrescento de uma nova função chamada *run*.

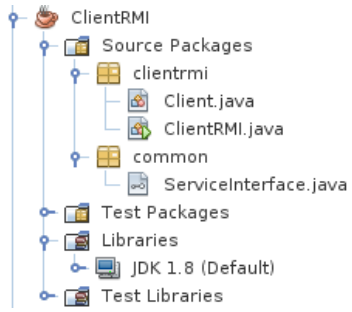


Figura 12 - NetBeans Project do Cliente RMI.

```
package clientrmi;

import common.ServiceInterface;
import java.rmi.NotBoundException;
import java.rmi.Remote;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.logging.Level;
import java.util.logging.Logger;

public class ClientRMI {

    public static void main(String[] args) {
        Client client = new Client();
        try {
            Registry registry =
        LocateRegistry.getRegistry("192.168.1.2",
        1234);
            Remote proxy = registry.lookup("Server
        Service");
            System.out.println("Connection
        successful!");
            ServiceInterface serviceInterface =
        (ServiceInterface) proxy;
            client.run(serviceInterface);
        } catch (RemoteException | NotBoundException
        ex) {
           Logger.getLogger(ClientRMI.class.getName()
        .log(Level.SEVERE, null, ex);
        }
    }
}
```

Código 10 – Cliente RMI e criação do Stub (Proxy).

```
package clientrmi;

import common.ServiceInterface;

public class Client {

    public static ServiceInterface serviceInterface;

    public void run(ServiceInterface s){
        serviceInterface = s;
        getObjectInvocationPerformanceAnalysis();
    }
```

Código 11 - Excerto do código Cliente RMI.

```
private static long getElapsedTimeOfObjectInvocation
(byte[] dataPackets) throws IOException{
     long elapsedTimeCounter = 0;
     for (int j=0; j < MAX_OBJECT_INVOCATIONS;
   j++){
           String data = gson.toJson(dataPackets);
           long startTime = System.currentTimeMillis();
           serviceInterface.process3(dataPackets);
           long stopTime = System.currentTimeMillis();
           long elapsedTime = stopTime - startTime;
           System.out.println(elapsedTime);
           elapsedTimeCounter = elapsedTimeCounter
        + elapsedTime;
     }
     System.out.println("");
     long averageElapsedTime =
   elapsedTimeCounter/MAX_OBJECT_INVOCATIONS;
     return averageElapsedTime;
}
```

Código 12 – Excerto do código Cliente RMI.

VI. Testes e Resultados

Após a implementação do código dos três Clientes, foi então possível registar a medição do tempo de invocação dos métodos das diferentes tecnologias. Para isso foram gerados arrays de tamanhos consecutivamente maiores e foram feitas 20 medições de modo a obter resultados mais fiáveis. Para obter as tabelas de valores usamos a função FileWriter, função incorporada no java, que permitiu geral automaticamente ficheiros com os valores esperados.

A. SOAP (XML)

1. Com Web Server

L. Banda vs Tamanho	32kpbs	64kpbs	128kpbs	1024kpbs	10000kpbs	100000kpbs	1048576kpbs
1	897	460	237	64	56	59	80
10	905	456	226	28	26	22	30
100	908	455	226	29	23	17	20
1000	922	463	232	31	21	16	16
10000	3889	1943	970	124	32	15	16
100000	35448	17711	8852	1105	132	30	22
1000000	369562	175339	87610	10949	1146	143	50

Tabela 1 – Largura de Banda vs Tamanho dos Pacotes (SOAP com Web Server).

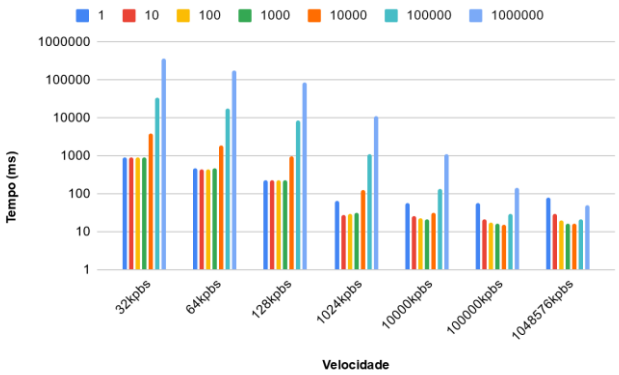


Figura 13 – Gráfico resultante da Tabela 1.

2. *Sem Web Server*

L. Banda vs Tamanho	32kpbs	64kpbs	128kpbs	1024kpbs	10000kpbs	100000kpbs	1048576kpbs
1	217	127	79	84	60	68	68
10	216	107	53	35	38	37	35
100	245	123	65	33	33	31	32
1000	563	282	139	37	30	35	28
10000	3711	1853	928	129	42	33	30
100000	35266	17607	8802	1117	140	42	36
1000000	368271	187432	89357	10959	1158	148	51

Tabela 2 - Largura de Banda vs Tamanho dos Pacotes (SOAP sem Web Server).

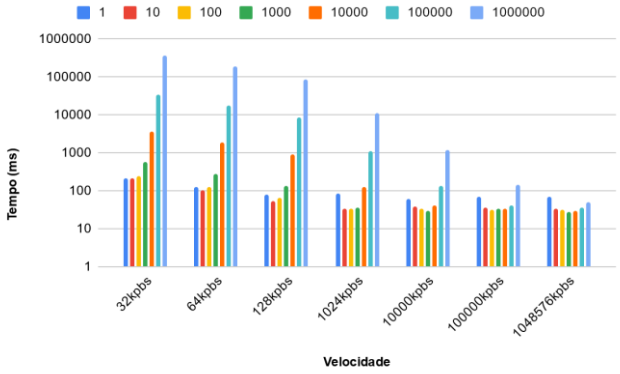


Figura 14 - Gráfico resultante da Tabela 2.

B. *REST (JSON)*

L. Banda vs Tamanho	32kpbs	64kpbs	128kpbs	1024kpbs	10000kpbs	100000kpbs	1048576kpbs
1	105	74	48	29	29	22	30
10	102	52	26	4	4	5	4
100	194	95	44	7	4	5	6
1000	1051	527	260	31	6	4	5
10000	9695	4838	2417	299	34	8	5
100000	102870	47911	23931	2989	310	39	11
1000000	1028700	484327	246325	31989	3080	332	67

Tabela 3 - Largura de Banda vs Tamanho dos Pacotes (REST).

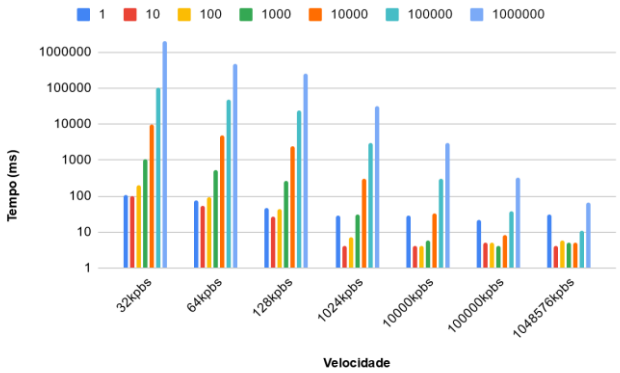


Figura 15 - Gráfico resultante da Tabela 3.

C. *RMI*

L. Banda vs Tamanho	32kpbs	64kpbs	128kpbs	1024kpbs	10000kpbs	100000kpbs	1048576kpbs
1	31	14	6	1	2	1	1
10	35	17	8	1	1	1	1
100	57	28	13	1	1	1	2
1000	323	136	67	7	2	1	0
10000	2645	1319	659	82	8	1	1
100000	26322	13155	6572	820	84	9	3
1000000	293257	152588	70309	8201	840	87	15

Tabela 4 - Largura de Banda vs Tamanho dos Pacotes (RMI).

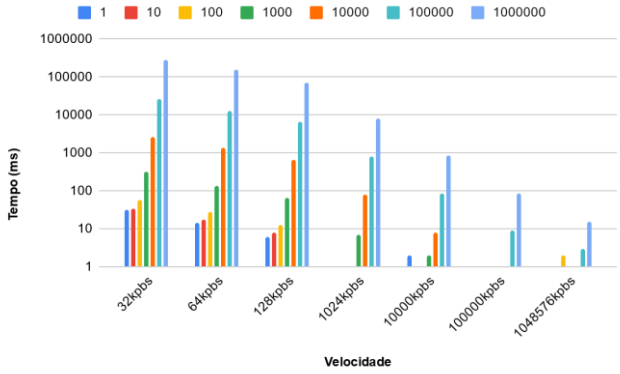


Figura 16 - Gráfico resultante da Tabela 4.

D. *SOAP vs REST vs RMI*

Método vs Tamanho	SOAP (com Web Server)	SOAP (sem Web Server)	REST	RMI
1	265	100	48	8
10	242	74	28	9
100	240	80	51	15
1000	243	159	269	77
10000	998	961	2471	674
100000	9043	9001	25437	6709
1000000	92114	93911	399260	75042

Tabela 5 - Método vs Tamanho dos Pacotes.

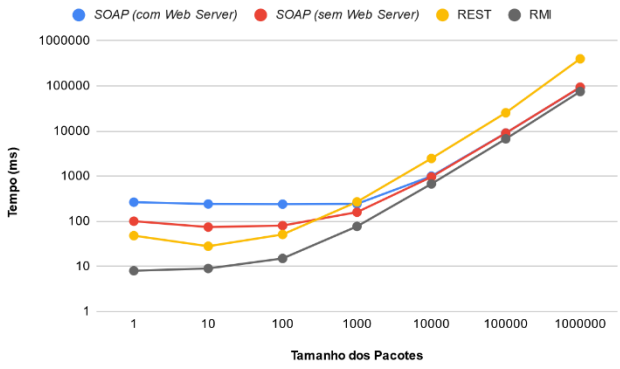


Figura 17 - Gráfico resultante da Tabela 5.

Método vs L. Banda	SOAP (com Web Server)	SOAP (sem Web Server)	REST	RMI
32kpbs	58933	58356	306102	46096
64kpbs	28118	29647	76832	23894
128kpbs	14050	14203	39007	11091
1024kpbs	1761	1771	5050	1302
10000kpbs	205	214	495	134
100000kpbs	43	56	59	14
1048576kpbs	33	40	18	3

Tabela 6 - Método vs Largura de Banda.

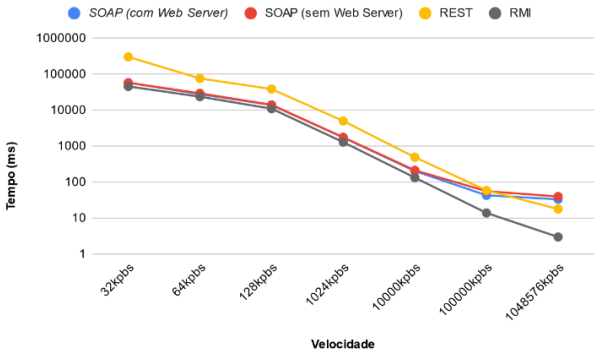


Figura 18 – Gráfico resultante da Tabela 6.

VII. Conclusão

Visto que foi possível testar as duas técnicas de *middleware* e as particularidades de cada uma, obteve-se resultados bastante satisfatórios.

Com a alteração do tamanho dos pacotes e da largura de banda disponível, verifica-se que a tecnologia mais eficaz é o *Java RMI*, depois o *Web Service SOAP* e em último lugar o *Web Service REST*.

Dentro do *Web Service SOAP* podemos ainda concluir que o método mais eficaz é sem *Web Server*, mas que a principal diferença ocorre entre o intervalo de 1 a 10000 em termos de tamanhos de pacotes, uma vez que a partir dos 10000 pacotes, os métodos têm uma performance bastante idêntica.

Como abordado em Unidades Curriculares anteriores, a tecnologia Java RMI foi a mais eficiente, tendo como única desvantagem o facto de ser exclusivamente Java e só correr em JVMs. Já entre o SOAP e o REST, os resultados foram mais parecidos e balanceados, onde até aos 1000 pacotes ganha o Web Service REST, mas a partir daí o SOAP mais eficiente.

Conclui-se então que, a seguir ao *Java RMI*, o SOAP é o protocolo de comunicação dos *Web Services* mais eficiente visto que a partir dos 1000 pacotes, o REST tem um aumento exponencial do tempo chegando a passar os 100000ms. De notar que, aquando da realização do teste do *Web Service REST*, com 100000 pacotes e com uma largura de banda de 32kpbs o valor teve que ser estimado, visto que o computador não aguentou a passagem acima dos 999999ms. Fazendo com que na nossa opinião seja o protocolo menos eficiente dos três.

Comprovou-se então que, o aumento exponencial do tamanho do array dos pacotes resulta num aumento exponencial do tempo de transmissão deste mesmo conjunto de dados e que o aumento da largura de banda resulta numa diminuição do tempo de forma quase proporcional.

VIII. Referências

[1] Mestre P. e Serôdio C., Gestão de Redes e Sistemas Distribuídos

[2] Mestre P., Serôdio C. e Oliveira Paula (2016/1017), Redes de Computadores

[3] https://pt.wikipedia.org/wiki/Web_service

[4] https://www.opensoft.pt/web-service/

[5] https://web.fe.up.pt/~eol/AIAD/aulas/JINIdocs/rmi1.html

[6] https://pt.wikipedia.org/wiki/RMI

[7]https://docs.oracle.com/javase/7/docs/technotes/guides/rmi/hello/hello-world.html

[8] https://www.w3schools.com